Section: CS

Understanding Prevalence of Malicious Advertising in Search Engine Results

Chris Humphreys

Problem Statement:

Malicious actors are abusing advertising infrastructure to deliver malware and related malicious content like ClickFix. While the problem is known to exist due to the large number of individuals who have fallen victim to these scams, there has been very little attempt to understand just how prevalent these malicious advertisements are in search engine results in the last roughly ten year time period. Prior to the ~2016 period, there were a number of documented attempts to understand the prevalence of these ads, but I have not been able to find any recent research on this topic. The problem this research will address is the problem of not having a clear picture of how prevalent malvertising currently is.

Solution Statement:

This research will attempt to measure the prevalence of malicious advertisements in search engine results in order to better understand both how adversaries are hiding malicious advertisements, and to try to understand how prevalent these ads are in search engine results. This is being done in order to address an area of study that has been neglected for some time. Additionally, this work can be built upon to create even better measures of the problem, or it can be used as a way to measure how effective various attempts to block or fix the malvertising problem are. It will also give us a measurement we can use against the older work to see if the problem of malvertising has increased or decreased since that work was undertaken.

Completed Tasks (Last 2 Weeks) – Public Gihub Repo:

- Reviewed my Oracles and assessed wether they were practical or not considering the work done up to this point. Made some changes to the oracles I am using and how I am using them. I tried to find a way in which I could submit the URLS to an online service to determine maliciousness. There are a couple solutions for this, but none that will really do what I want in a realistic time-frame. (At least without paying). The two largest contenders were CloudFlare and VT, but both really suffer from a latency problem in that if I have to submit each URL I wont be able to scan nearly the number I would like (~10k). And both will likely block me based on the volume of requests.
- In response to the above, I am slightly changing my goal. My goal now is to see if it is possible to identify malicious google ads using static lists and local information (ie. SSL Valid From dates) to check against. I have the URLHaus URL List, and the github “malicious-domains” list for checking against. These update at most hourly so are as accurate as can be found.
- Wrote the code that brings in and updates the URLHaus list.
- Completely reworked by Detector class to make it more accurate. Before I was using the python requests library which cannot execute java-script, so any js redirection would not have been followed. I ended up completely replacing this with the python playwright library which uses actual browsers to go to the sites and follow redirects, including js based ones.
- Made my requests much quicker and less resource intensive by blocking all requests for media (images, videos, etc.) from the ad pages. Now it pull just the text and few other things.

- Completely reworked the multi-threading in my detector class. The playwright library does not support multi-threaded execution, but it does off an asynchronous version. While I found it extremely challenging to implement this, I did learn a ton from it, and it is now running asynchronously and I am able to limit the number running concurrently via a semaphore.
- Added the 'playwright-stealth' library which should further improve my chances of getting past malicious ad cloaking.
- Wrote the malicious url list oracle. This pulls a list of known malicious urls from the URLHaus database, and then compares my list of final ad pages to this list of known bad urls.
- Wrote the SSL Oracle. When playwright access an ad page, I have it pull the ValidFrom date from the SSL cert. Then check to see if the date is less than 24 hours old. If it is, it is assumed to be suspiciously young and will mark. In my limited testing I have found most legitimate certs to be substantially older than 24 hours, but when I start getting results will have to assess what the final time frame to use is.

Coming up these next two weeks is to just write the wrapper to capture my results, which should be relatively easy, and then start doing my actual runs. I plan on slowly increasing the number with the goal of being able to run against a full 10,000 search engine page results. One big change I am still contemplating is whether to continue pulling the full text of the adds. Initially I was doing this to search for click-fix related strings, but with my existing Oracles, I'm thinking this may be kind of pointless. Without it, I think I could just make a HEAD request and get my results much quicker. Honestly, I think I may not have a choice on this, at least if I want to be able to run this against more than ~1,000 ad results. I think I am going to leave now, and then depending on how well the testing goes this week make the final decision. Also going to start writing the first drafts of the paper this week.

One last item I will work on this week (and have added to my tasklist) is reviewing the success criteria of some of the other work done like this and see if I can borrow from that to help in further development of my own Evaluation.

Questions I have or Issues I'm running into:

Big question for this week is whether to keep the full text of these web pages coming in, or just go with HEAD requests. It is going to save a ton of time and space to not to pull the full text of each site.

The other major concern I have is that I'm not going to find any ads. I think I have really maximized my chances, but until I run these things I really don't know if its going to work.

Methodology Paragraph Summary:

My proposed methodology is to use a (Search Engine Results Page) API provider to collect advertisement URLs from Google. After collecting the advertisement URLs, I will use a residential proxy service in conjunction with a "collector" and "detector" program which I will write to collect the contents of the sites linked to from the ads. Then I will use oracles based on the ones used in the 2014 paper "The Dark Alleys of Madison Avenue: Understanding Malicious Advertisements"[18] to determine if the site is malicious or not. I will create a "Classifier" class to handle this aspect. The results and number of malicious ads identified can be compared with the results of previous studies to better understand the prevalence of modern malvertising.

One concern I have is that many of these malicious ads may use "cloaking" in which if they detect a scraper or researcher they send the user to a benign site rather than the malicious one. In the 2015 paper "Automated Collection and Analysis of Malware Disseminated via Online Advertising"[16] a model is

suggested for a “Detector” of malicious advertisements that opens pages to determine if the URL leads to a file download. I plan on borrowing this model to create a similar detector which I will run over the results of my search engine scrape. In addition to looking for file downloads, I will also look for strings indicative of ClickFix or other malicious content. Additionally, I will use some sort of residential proxy for these requests to make it more difficult for malicious advertisements to realize I am a researcher and send me to an otherwise benign site.

Timeline:

Week #	**Description of Task**	**Status**
W1 (Aug 25 to Aug 31)	Brainstorm Ideas for Project → Firefox Add On	Completed
W1	Research tutorials and lessons on building Firefox AddOns	Complete
W1	Build borderify sample AddOn from Firefox documentation	Complete
W1	Build Beastify AddOn from Firefox documentation	Complete
W2 (Aug 31 to Sept 6)	Brainstorm and determine best course of action based on initial feedback from Dr. Ahmad	Complete
W2	Search ACM articles for anything related to malvertising, read pertinent articles and updated proposal based on findings.	Complete
W2	Search IEEE for any articles related to malvertising, read pertinent articles and update proposal based on findings.	Complete
W2	Do a web search engine based search for non-academic research and findings regarding malvertising.	Complete
W2	Synthesize all findings from literature review and then update proposal and problem statement based on findings.	Complete
W2	Review Dr. Ahmad’s feedback again in relation to updated proposal and problem statement ideas. Update to include all feedback.	Complete
W3 (Sept 7 to Sept 14)	Determine which SERP is best suited for my task, set up account and test functionality	Complete
W3	Determine which residential proxy is best suited for my task, set up account and test functionality	Complete
W3	Find a modern working alternative for wepawet oracle. Set up account and test functionality	Complete
W3	Make skeleton of “collector” code that will utilize the SERP API to pull in search engine results.	Complete
W4 (Sept 14 to Sept 20)	Complete code for “Collector” and have it fully functional and ready to be used.	Complete
W4	Find one or more “Domain blacklist” sites I can use to compare ads against to determine if malicious.	Complete
W5 (Sept 21 to Sept 27)	Make skeleton of Detector code.	Complete
W5	Integrate residential proxy with detector code so	Complete

	that requests are less likely to be identified by adversaries	
W6 (Sept 28 to Oct 4)	Complete Detector code and ensure it has the ability to read from the Ad URL's as expected	Complete
W7 (Oct 5 to Oct 11)	Combine detector and collector code into a single application	Complete
W7	Add capability that allows findings to be compared against the "blacklisted Domain Oracle"	Complete
W8 (Oct 12 to Oct 18)	Add capability that allows findings to be compared against the "SSLTimeFrame" malware hash Oracle.	Complete
W8	Add capability that allows findings to be compared against the "URLHaus URL List"	Complete
W9 (Oct 19 to Oct 25)	Combine all Oracle and previous code into one cohesive application that can be used to start scanning.	Incomplete
W9	Review literature for Evaluation Criteria I can borrow for my own project.	Incomplete
W9	Do at least 1 successful test run of 1000 pages of search engine results,	Incomplete
W10 (Oct 26 to Nov 1)	Based on results from W9 test run, update code to solve any identified issues	Incomplete
W11 (Nov 2 to Nov 8)	Based on test run from W9, determine ideal number of runs to use.	Incomplete
W11	Run a scan based on number of times decided on above.	Incomplete
W11	Vet results of full run looking for code errors, bugs etc. Fix any identified issues	Incomplete
W12 (Nov 9 to Nov 15)	Do true scan, the results of which will be the final results presented in the paper. – Ensure done by 15th so that I can make presentation video.	Incomplete
W13 (Nov 16 to Nov 22)	Analyze findings, determine conclusions, compare against results from earlier studies, finish first draft of final paper	Incomplete
W13	Complete second draft of Final Paper.	Incomplete
W14 (Nov 23 to Nov 29)	Thanksgiving	
W15 (Nov 30 to Dec 6)	Write final draft of paper	Incomplete
W15	Submit!	Incomplete

Evaluation:

I don't want to base my evaluation on how many malicious ads I find, as that number could be low for a number of reasons beyond my control. Instead, I think I should measure my success by the number of ad pages I am able to evaluate, and then how accurate that assessment turns out to be.

Success for this project would look like being able to evaluate 10k search engine result pages against my Oracles and successfully labeling any ads which are malicious as being such. That of course begs the

question of how will I know how accurate I was. I think my plan is to grab 100 random analyzed ad pages, that have been through my oracles, and manually check the findings by hand. I can compare this to the number my Oracles identified as malicious and get an accuracy percentage, from that I will extrapolate it and use that as my total accuracy. Knowing this plan, I think the evaluation criteria below is reasonable.

- # of Ads Evaluated
  - < 500 Poor
  - 500 – 1000 Adequate
  - 1000 – 5000 Good
  - 5000 – 10000 Great
  - >= 10000 Excellent
- Accuracy %
  - < 50% Poor
  - 50% – 70% Adequate
  - 70% - 90% Good
  - 90% - 99% Great
  - 100% Excellent

Report Outline:

Draft Report (Right Now my draft is basically my outline: This will flesh out as time passes):

I. Abstract:

II. Introduction:

A. Define Problem (Lack of studies on search engine malicious ad prevalence since 2016)

B. Clearly State Negative repercussions of not studying this; orgs and individuals are targeted and victimized.

III. Past Research

A. Review previous studies that directly mirror what I will be doing or that I borrowed and built up the methodology

1. The Dark Alleys of Madison Avenue: Understanding Malicious Advertisements.

a. Borrowed Idea of Oracles and took inspiration for some of mine

2. Automated Collection and Analysis of Malware Disseminated via Online Advertising

a. this is where I will take my search queires from. They also have some system design items I took inspiration I will elaborate on

B. Review Related malvertising studies that are less directly related
    1. All the ones that use machine learning or novel methods of identification
    2. Articles that look for malicious ads, but in different contexts like browser extensions
C. Discuss the relative lack of work over the past 5 to 10 years

IV. Methodology
A. SERP and Pulling google Ads
    1. Issues with rate limiting
    2. SERP providers and other options
    3. Discussion of my decision to use SERP and why
B. Residential proxy
    1. Cloaking and discussion of why a proxy is even necessary
    2. Other options and why I went with a commercial proxy
    3. Issues not resolved by proxy usage
C. Query selection process
    1. discuss previous work and what I copied
    2. discuss changes I made and why
        a. Clickfix searching
        b. New technologies and terms since last paper
D. System Design
    1. Review of how my system works
        a. Python object “Collector”, collects the google ad links
        b. Python object “Detector”, detects the actual ad website, ideally bypassing cloaking protections
        c. Python object “Classifier”, runs the oracles which determine if the ads are malicious or not.
            i. Domain BlockList Oracle
            ii. URLHaus Oracle
            iii. SSL Oracle

V. Results
A. Findings

1. After a run of ideally about 10k queries (may need to adjust based on speed) how many ads were identified
2. Of identified ads how many were deemed malicious and by which oracle
    a. Domain Oracle Results
    b. SSL Oracle Results
    c. URLHaus Oracle Results
3. Optional: Findings at different times and locations

B. Comparisons to previous work
1. What findings did other papers have, and even if the match is not 1:1, can we infer findings from what we do see.
2. Highlight significant deviations from previous work and attempt to explain.

C. Future Work
1. Better ways to avoid cloaking
2. How to build on this work

VI. Conclusion
A. Did I find ads, and how many
B. Was this worthwhile, and more importantly, is it worth continuing, or did I learn that this method of detection is not longer valuable.

References:

[1] anon. Malvertising What is malvertising and how to protect yourself. Malwarebytes. Retrieved September 65 2026 from https://www.malwarebytes.com/malvertising

[2] anon. 2026. Malvertising & Ad Quality Index - Risky Business. Retrieved September 6, 2026 from https://www.confiant.com/hubfs/Team%20-%20Marketing/THE%20MAQ%20INDEX%202026_%20RISKY%20BUSINESS.pdf

[3] Contributors to Wikimedia projects. 2010. Malvertising. Wikipedia. Retrieved September 6, 2026 from https://en.wikipedia.org/wiki/Malvertising

[4] Gadget Hacks. 2026. Ad Blockers That Still Work in Chrome: 4 Ways to Block Ads. Gadget Hacks. Retrieved September 6, 2026 from https://android.gadgethacks.com/how-to/ad-blockers-that-still-work-in-chrome-4-ways-to-block-ads/

[5] David Kasabji. 2025. From SEO Poisoning to Malware Deployment: Malvertising campaign uncovered. Conscia. Retrieved September 5, 2026 from https://conscia.com/blog/from-seo-poisoning-to-malware-deployment-malvertising-campaign-uncovered/

[6] Jérôme Segura. 2016. A Look Behind The Skype Malvertising Campaign | Malwarebytes Labs. Malwarebytes. Retrieved September 5, 2026 from https://www.malwarebytes.com/blog/news/2016/02/a-look-behind-the-skype-malvertising-campaign

[7] Dhanur Sehgal. 2026. What Is Malvertising? Examples, Risks & Prevention. The Blockify Blog. Retrieved September 5, 2026 from https://getblockify.com/blog/malvertising/

[8] Michael Steele. 2026. SourTrade: Browser-Assembled Malware Delivered Through Malvertising. Confiant. Retrieved September 5, 2026 from https://blog.confiant.com/p/sourtrade-browser-assembled-malware

[9] George Thomas. 2015. Malvertising up 325% - Are the AdBlockers Working?. Retrieved September 6, 2026 from https://web.archive.org/web/20160130161632/http://blog.check-and-secure.com/091015-malvertising-up-325-are-adblockers-working/

[10] Shubham Kumar, Siddharth Swarup Rautaray, and Manjusha Pandey. 2017. Malvertising: A case study based on analysis of possible solutions. In 2017 International Conference on Inventive Computing and Informatics (ICICI), November 2017. IEEE, 288–291. https://doi.org/10.1109/icici.2017.8365356

[11] Joshua Rolon, Cheryl Hinds, and Felicia Doswell. 2019. SpartanShield: A Layered Defense against Malvertising. In 2019 International Conference on Computational Science and Computational Intelligence (CSCI), December 2019. IEEE, 185–190. https://doi.org/10.1109/csci49370.2019.00038

[12] Eric Blancaflor, Jon Eril R. Aparize, Joyous M. Geronimo, Ethan Joseph S. Gonzales, and Joyful L. Morilla. 2024. Malvertising in AI-Powered Chat Platforms: Utilizing Integrated Artificial Intelligence Solutions for Threat Detection and Mitigation. In 2024 8th International Conference on Communications and Future Internet (ICCFI), July 19, 2024. IEEE, 67–74. https://doi.org/10.1109/iccfi63302.2024.00018

[13] Ankita Kumari and Ishu Sharma. 2024. Mitigating Malvertising Threats: An Exploration of Machine Learning Classification Algorithms for Effective Detection. In 2024 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC), January 27, 2024. IEEE, 1–6. https://doi.org/10.1109/assic60049.2024.10508033

[14] Chin-Tser Huang, Muhammad N. Sakib, Charles Kamhoua, Kevin Kwiat, and Laurent Njilla. 2017. A game theoretic approach for inspecting web-based malvertising. In 2017 IEEE International Conference on Communications (ICC), May 2017. IEEE, 1–6. https://doi.org/10.1109/icc.2017.7996807

[15] Chin-Tser Huang, Muhammad N. Sakib, Charles A. Kamhoua, Kevin A. Kwiat, and Laurent Njilla. 2020. A Bayesian Game Theoretic Approach for Inspecting Web-Based Malvertising. IEEE Trans. Dependable and Secure Comput. 17, 6 (November 2020), 1257–1268. https://doi.org/10.1109/tdsc.2018.2866821

[16] Muhammad N. Sakib and Chin-Tser Huang. 2015. Automated Collection and Analysis of Malware Disseminated via Online Advertising. In 2015 IEEE Trustcom/BigDataSE/ISPA, August 2015. IEEE, 1411–1416. https://doi.org/10.1109/trustcom.2015.539

[17] Tiliang Zhang, Hua Zhang, and Fei Gao. 2013. A Malicious Advertising Detection Scheme Based on the Depth of URL Strategy. In 2013 Sixth International Symposium on Computational Intelligence and Design, October 2013. IEEE, 57–60. https://doi.org/10.1109/iscid.2013.128

[18] Apostolis Zarras, Alexandros Kapravelos, Gianluca Stringhini, Thorsten Holz, Christopher Kruegel, and Giovanni Vigna. 2014. The Dark Alleys of Madison Avenue: Understanding Malicious Advertisements. In Proceedings of the 2014 Conference on Internet Measurement Conference, November 05, 2014. ACM, 373–380. https://doi.org/10.1145/2663716.2663719

[19] Xinyu Xing, Wei Meng, Byoungyoung Lee, Udi Weinsberg, Anmol Sheth, Roberto Perdisci, and Wenke Lee. 2015. Understanding Malvertising Through Ad-Injecting Browser Extensions. In Proceedings of the 24th

International Conference on World Wide Web, May 18, 2015. International World Wide Web Conferences Steering Committee, 1286–1295. https://doi.org/10.1145/2736277.2741630

[20] Zhou Li, Kehuan Zhang, Yinglian Xie, Fang Yu, and XiaoFeng Wang. 2012. Knowing your enemy. In Proceedings of the 2012 ACM conference on Computer and communications security, October 16, 2012. ACM, 674–686. https://doi.org/10.1145/2382196.2382267

Appendix

If there are notes, sources, figures you want to reference, or your draft writing please include that here.